CRAFT Linux Operating Modes and Tickets

Certificate Request Agent for Tickets | Process overview

CRAFT obtains a user's Kerberos ticket-granting ticket (TGT) through PKINIT from Linux, including hosts that are not domain-joined. It supports three operating modes. Linux can orchestrate privileged certificate enrollment, orchestrate privileged credential linking, or use a certificate exported from Windows with unprivileged acquisition. All three publish the user's cache for unattended applications without storing the user's AD password.

Choose an operating mode

Mode	Credential source and authority	Setup
1 Privileged enrollment	Linux generates a user key and enrolls a certificate through CES and AD CS with an enrollment-agent signature. The CA restricts the agent, template and recipients.	Mode 1 setup
2 Privileged credential linking	Linux adds a temporary public key to the user's msDS-KeyCredentialLink and uses Key Trust PKINIT. AD attribute delegation defines the target scope.	Mode 2 setup
3 Unprivileged Windows export	Windows enrolls and exports a certificate and matching private key. Linux uses the securely transferred PEM pair under the caller's UID.	Mode 3 setup

How CRAFT selects the mode

A complete pair at the real caller's NSS-home ~/.config/craft/user.pem and user.key takes priority on every installation. Partial, unsafe, expired, revoked or invalid inputs fail and preserve the cache. Only when both files are absent does mechanism=enrollment select mode 1 or mechanism=key_trust select mode 2. A failed privileged mechanism never switches to the other one.

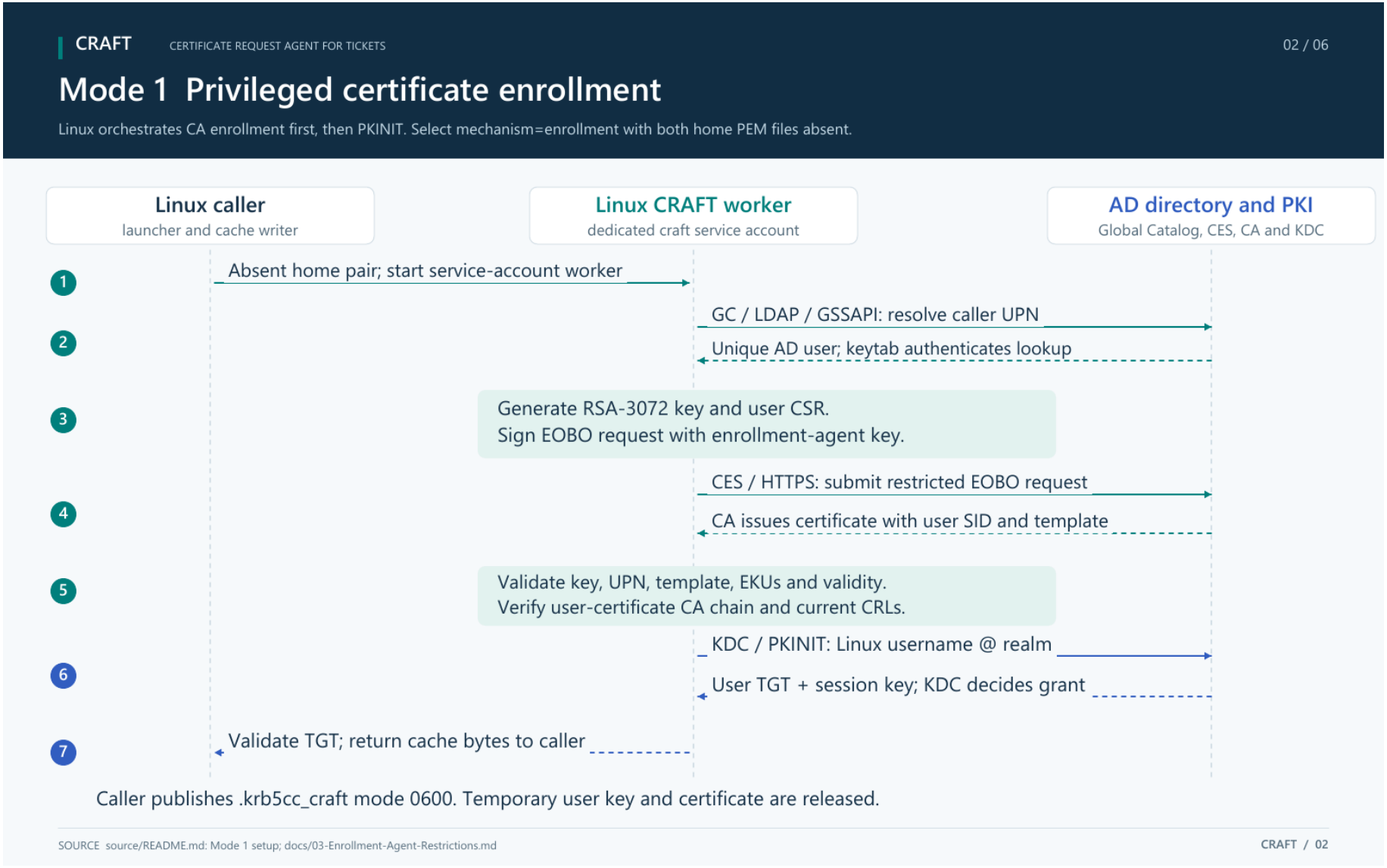
Mode 3 uses ordinary executables. Modes 1 and 2 require the installed setuid launcher, dedicated craft service account, craft-users caller group and private /run/craft runtime directory. Users invoke craft directly as themselves, without sudo. Certificate and network processing run outside root, and cache publication runs as the caller.

Three-mode selection diagram | Build and install

CRAFT supports automation alongside MFA-protected sign-in, but does not perform an MFA challenge or establish that one was completed. The next pages show each acquisition flow; the final pages explain identity, storage and the shared ticket lifecycle.

Mode 1 privileged certificate enrollment

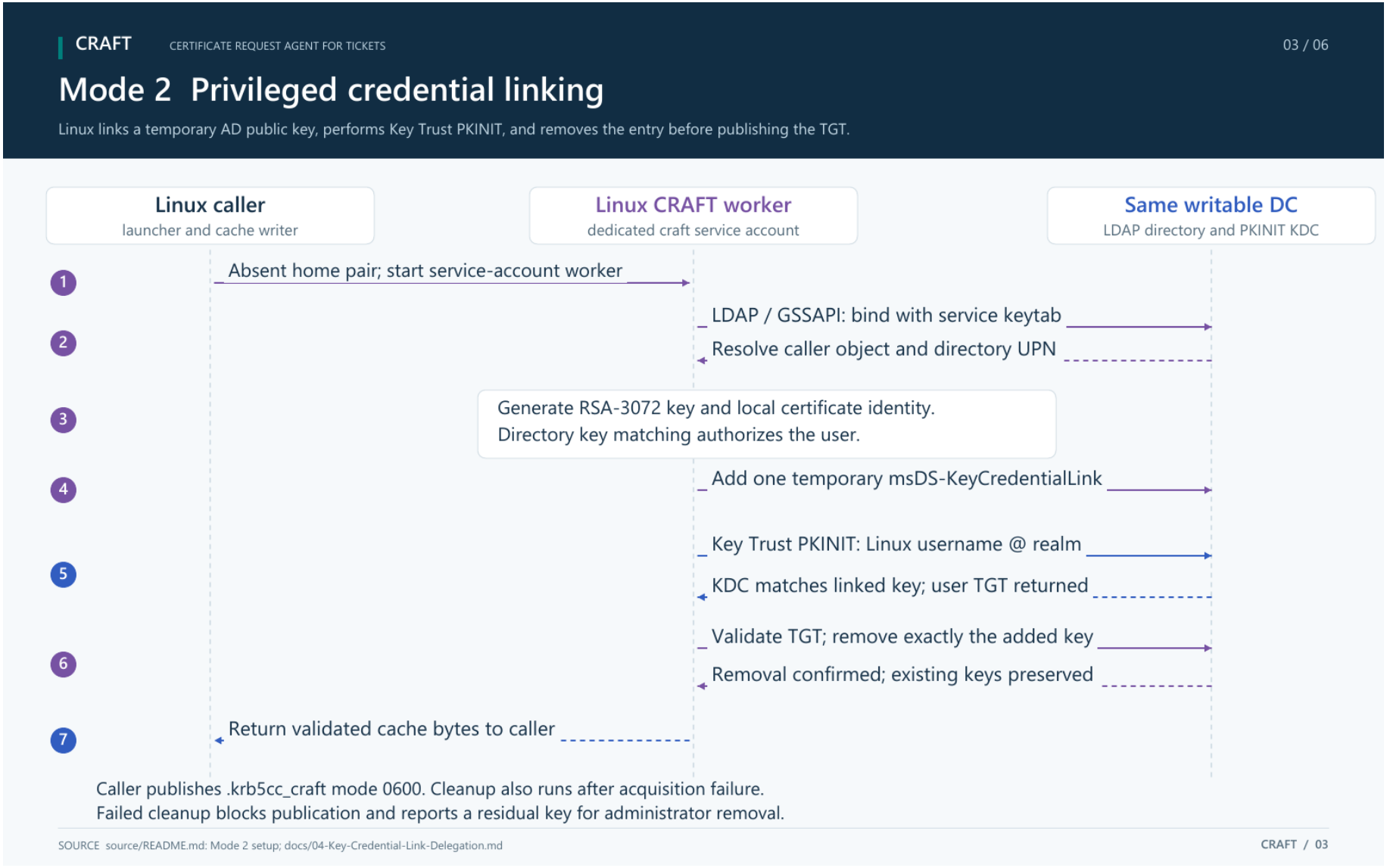
Complete setup instructions | Editable flow diagram



Linux resolves the user, enrolls a restricted certificate through CES and AD CS, then obtains the user's TGT through PKINIT. The generated user pair is temporary; CA records remain.

Mode 2 privileged credential linking

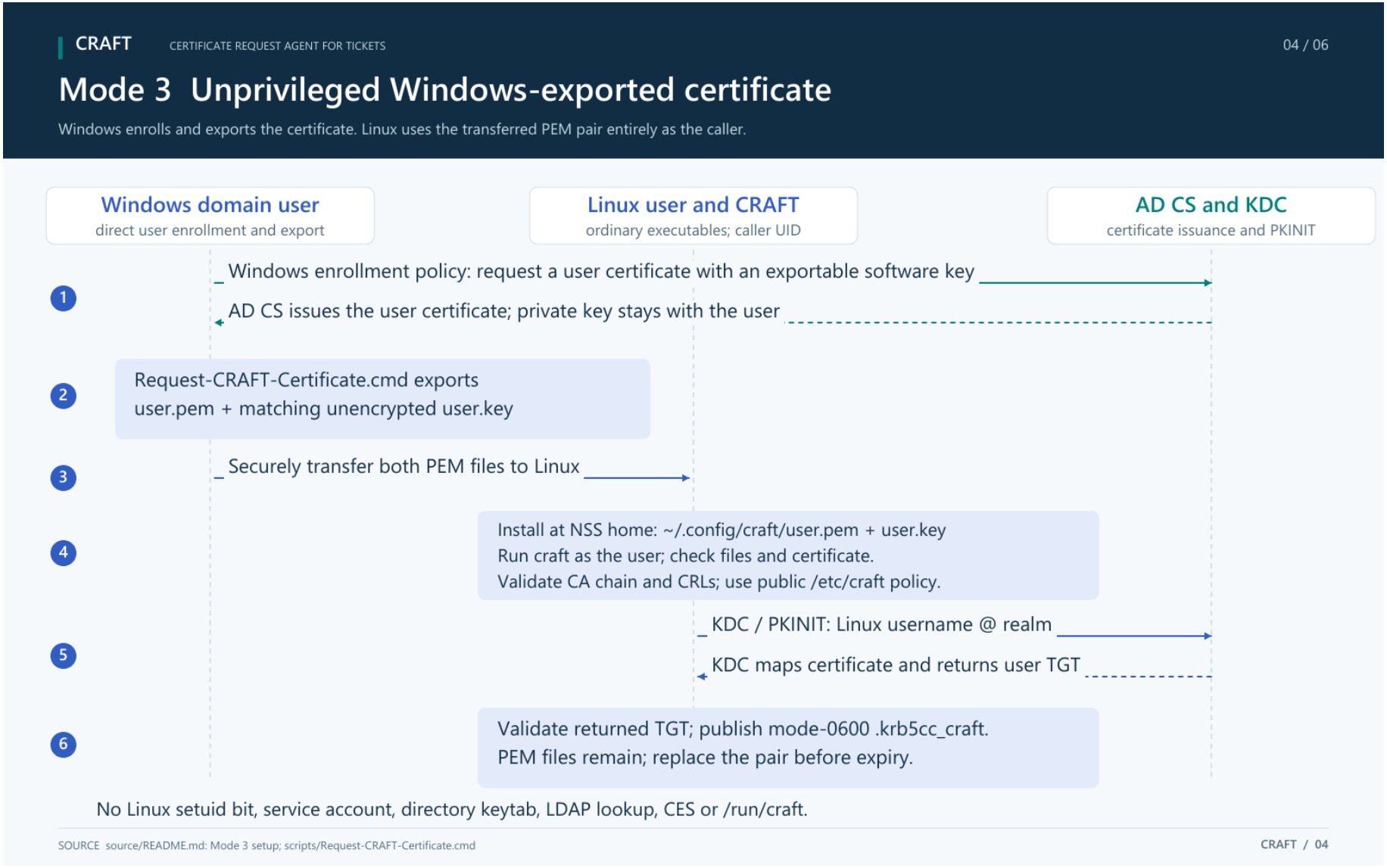
[Complete setup instructions](#) | [Editable flow diagram](#)



Linux uses a delegated directory write and Key Trust PKINIT against the same writable DC. Cleanup must succeed before cache publication. A residual key after an outage needs administrator removal.

Mode 3 unprivileged Windows exported certificate

Complete setup instructions | Editable flow diagram



Windows enrollment and export precede secure PEM transfer. Linux validates the supplied pair and obtains the user's TGT as the caller. The PEM pair remains and must be replaced before expiry.

Identity trust and credential storage

All modes request <Linux username>@<configured realm> and verify the returned TGT principal, flags, AES encryption and lifetimes. Linux names must match the intended AD sAMAccountName. Enrollment and credential linking resolve and check the directory UPN. The supplied-certificate path performs no LDAP lookup; the KDC must map that certificate to the fixed caller principal. The certificate CN cannot select another account.

Mode	Client credential and authorization	What persists
1 Enrollment	CA-issued user certificate; matching key, UPN, template, three client EKUs, short validity, CA chain and CRLs. CA recipient restrictions authorize issuance.	Private agent/keytab/transport credentials stay under /etc/craft. CA issuance and audit records remain. Generated user key and certificate are released.
2 Credential linking	Locally issued certificate carrying a fresh RSA-3072 public key. The KDC matches the linked key; a trusted user-certificate CA chain is unnecessary. KDC trust is still required.	Service keytab and delegated AD rights remain. CRAFT removes its exact directory entry and releases the generated identity. Failed cleanup can leave a standing credential.
3 Supplied certificate	Windows-exported PEM pair; matching key, logon usage, UPN SAN, validity, CA trust and CRLs. KDC account mapping authorizes authentication.	user.pem and user.key stay in the caller's home. CRAFT does not renew, overwrite or delete them. Replace them before expiry.

Every mode atomically publishes a caller-owned mode-0600 .krb5cc_craft in the NSS home. This persistent cache contains the TGT and session key, without the user's certificate or private key. Applications use it to request service tickets. Success replaces the cache; acquisition failure preserves it, and expiration does not delete it automatically.

Generated credentials and sealed PKINIT memory files are released after the operation. Memory can reach swap or privileged host capture. Deleting keys, removing a linked credential, ending a process or removing a cache does not revoke an already-issued ticket. Host and service-account compromise expose the authority granted to that installation.

Enrollment-agent restrictions | Key Trust delegation

Ticket lifetimes and everyday use

CRAFT requests a ten-hour initial TGT and a seven-day renewal window with AES256/AES128. The KDC controls the grant; certificate/key lifetime can constrain initial validity. A renewal window is the period during which an eligible TGT can be renewed. It does not make one ticket valid for seven days.

require_full_tgt_lifetime=yes rejects shortened initial or renewal grants without replacing the cache. Administrators can set no to accept shorter grants with a warning. Returned ticket times are never rewritten. Enrollment certificate caps default to ten hours total, including backdating; they do not cap mode 3's supplied certificate or mode 2's local identity.

Select the user cache

```
cache=$(/usr/local/bin/craft) || exit "$?"
export KRB5CCNAME="$cache"
klist -ef -c "$KRB5CCNAME"
```

Kerberos-aware applications launched from this shell use the selected cache for SMB, LDAP and Kerberos-enabled web services. The service still enforces the user's permissions. CRAFT cannot change its parent shell's environment; the optional with-craft wrapper runs a command with its cache selected.

Maintain a long running job

```
job_pid=$$
cache=$(/usr/local/bin/craft-maintain --watch-pid "$job_pid") || exit "$?"
export KRB5CCNAME="$cache"
```

craft-maintain returns when credentials and detached maintenance are ready. Renewal uses the cached ticket and session key alone. Fresh authentication near the absolute renewal deadline invokes craft: reuse the supplied pair, or use the configured enrollment or credential-linking mode when both files are absent. Full ten-hour/seven-day grants normally renew near hour eight and authenticate afresh near day six; actual grants set the schedule.

Install the maintainer without setuid/setgid on Linux with /proc and system Kerberos configuration matching the pinned realm/KDC and AES policy. Monitoring uses pidfds when available and /proc elsewhere. Home refresh works under no_new_privs; modes 1 and 2 require later setuid calls. The scheduler must retain background processes. Watch a shell or job controller that lasts for the job; optional --max-duration 21d adds a cutoff. Concurrent jobs for one UID share the cache and fresh-acquisition cooldown.

```
/usr/local/bin/craft-maintain --status --watch-pid "$job_pid"
klist -ef -c "$KRB5CCNAME"
```

Failures preserve credentials and retry; later failures do not stop the job. Status and AUTHPRIV logs report failures. Maintenance stops with the watched PID and leaves the cache. Applications must reload refreshed credentials. Keep CRLs current and replace the supplied certificate before expiry; TGT maintenance does not renew it.

Maintenance setup and status | Credential timing diagram

References and license

Source and setup | Security review | Acceptance checks

PKINIT specification | MIT PKINIT configuration

Source code, scripts, configuration examples and documentation are licensed under GNU GPL version 3 only (GPL-3.0-only). See LICENSE for the terms and warranty disclaimer.